

# Plano de Aula

# Plano de Aula – Persistência de Dados com JSON no Flutter

**Curso:** Desenvolvimento Mobile com Flutter

**Disciplina:** Programação Mobile

**Aula:** Uso de JSON em Flutter – Do Local à Integração com API

**Pré-requisitos:** Noções básicas de Flutter, Dart, componentes de UI e manipulação de dados

**Recursos Necessários:** VSCode/Android Studio, Android Emulator ou dispositivo real, internet, bibliotecas Flutter (shared\_preferences, path\_provider, http)

---

## Objetivos da Aula

- Compreender a estrutura e função do formato JSON.
  - Utilizar JSON para persistência de dados locais com shared\_preferences.
  - Armazenar e recuperar dados em arquivos locais com path\_provider.
  - Realizar leitura e escrita de arquivos .json.
  - Consumir dados de APIs REST usando JSON com o pacote http.
  - Desenvolver um projeto prático completo com o uso de JSON (cadastro, listagem e integração com API).
- 

## Conteúdo Programático

| Conteúdo                                    | Descrição                                                   |
|---------------------------------------------|-------------------------------------------------------------|
| 1. Introdução ao JSON                       | Estrutura, chaves e valores, exemplos práticos              |
| 2. JSON com SharedPreferences               | Armazenamento de pequenas configurações e preferências      |
| 3. JSON com Arquivos Locais (path_provider) | Criação, leitura e escrita de arquivos .json no dispositivo |

|                                                                     |                                                                                  |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>4. Serialização/Deserialização com <code>dart:convert</code></b> | Conversão de objetos para JSON e vice-versa                                      |
| <b>5. Consumo de API com JSON</b>                                   | Uso do pacote <code>http</code> , requisições GET/POST, parsing de resposta JSON |
| <b>6. Projeto prático completo (MVC)</b>                            | Cadastro de produtos com imagem, persistência local e consulta de dados externos |

---

## Metodologia (Estratégias de Ensino)

- Exposição dialogada com slides e código comentado
- Demonstração prática com projeto real em Flutter
- Atividade em duplas: implementar funcionalidades com JSON
- Discussão orientada sobre vantagens, limitações e boas práticas

---

## Atividades Práticas

### Atividade 1 – Introdução e Visualização de JSON (30 min)

- Análise de um arquivo JSON simples (manual ou online)
- Criar um Map e converter para JSON usando `json.encode()`
- Converter JSON em Map com `json.decode()`

```
Map<String, dynamic> dados = {'nome': 'Ana', 'idade': 28};  
String jsonString = json.encode(dados); // Converte para JSON  
Map<String, dynamic> novoMapa = json.decode(jsonString); // De volta  
para Map
```

---

### Atividade 2 – Shared Preferences com JSON (30 min)

- Armazenar preferências do usuário (nome, tema, idade)
- Carregar automaticamente ao iniciar o app

```
SharedPreferences prefs = await SharedPreferences.getInstance();  
prefs.setString('user', json.encode({'nome': 'Lucas', 'idade': 21}));
```

---

### Atividade 3 – Persistência com Arquivo Local (45 min)

- Criar arquivo JSON e armazenar lista de objetos
- Manipular dados com File e path\_provider

```
Directory dir = await getApplicationDocumentsDirectory();  
File file = File('${dir.path}/produtos.json');  
await file.writeAsString(json.encode(listaProdutos));
```

---

### Atividade 4 – Projeto Prático (1h 30min)

**Tema:** Cadastro e listagem de produtos (nome, preço, categoria, imagem)

**Funcionalidades:**

- Tela de cadastro de produto com campos e botão para tirar foto
- Serialização dos produtos em arquivo JSON
- Leitura e exibição da lista cadastrada
- Estrutura de projeto em **MVC**
- Upload de foto com image\_picker
- Persistência de dados com path\_provider

---

### Atividade 5 – Integração com API REST (45 min)

- Criar tela de consulta de produtos em uma API pública (ex: JSONPlaceholder, RESTFake, API própria)
- Utilizar `http.get()` para consumir os dados
- Exibir produtos usando `ListView.builder`

```
Future<List<dynamic>> fetchProdutos() async {  
  final response = await  
http.get(Uri.parse('https://api.exemplo.com/produtos'));  
  if (response.statusCode == 200) {  
    return json.decode(response.body);  
  } else {  
    throw Exception('Erro ao carregar dados');  
  }  
}
```

---

### Recursos Didáticos

- Slides sobre JSON e estrutura de dados
  - Repositório GitHub com códigos-base e desafio
  - Links úteis:
    - [https://pub.dev/packages/shared\\_preferences](https://pub.dev/packages/shared_preferences)
    - [https://pub.dev/packages/path\\_provider](https://pub.dev/packages/path_provider)
    - <https://pub.dev/packages/http>
    - [https://pub.dev/packages/image\\_picker](https://pub.dev/packages/image_picker)
-

## Avaliação

| Critério                                    | Descrição                                | Peso |
|---------------------------------------------|------------------------------------------|------|
| Participação nas atividades                 | Entendimento dos conceitos               | 30%  |
| Implementação do projeto local (MVC + JSON) | Código funcional, arquitetura            | 40%  |
| Integração com API                          | Capacidade de manipular JSON de resposta | 30%  |

---

## Extensões Futuras (Aulas Posteriores)

- Integração com Firebase e uso de Firestore (JSON nativo)
- Integração com banco de dados relacional usando JSON intermediário

# 1 - Introdução ao Json

# 1. Introdução ao JSON

**Tópico:** Estrutura, chaves e valores + Exemplos práticos

## O que é JSON?

**JSON** (JavaScript Object Notation) é um formato **texto leve, estruturado**, usado para **armazenar e transportar dados**. Ele é baseado em JavaScript, mas pode ser usado por praticamente todas as linguagens, incluindo Dart (utilizado no Flutter).

---

## Estrutura Básica do JSON

Um objeto JSON é composto por **pares chave:valor**.

```
{  
  "nome": "Maria",  
  "idade": 25,  
  "ativo": true  
}
```

### Regras:

- **Chaves** (keys): sempre entre aspas duplas (" "), e devem ser strings.
- **Valores** podem ser:
  - String: "Texto"
  - Número: 25
  - Booleano: true / false
  - Array (lista): [ ]
  - Objeto (outro JSON): { }



- Null: null

---

## Tipos de Dados em JSON

| Tipo     | Exemplo                                  |
|----------|------------------------------------------|
| String   | "cidade": "Limeira"                      |
| Número   | "idade": 30                              |
| Booleano | "ativo": true                            |
| Array    | "hobbies": ["nadar", "ler"]              |
| Objeto   | "endereco": { "rua": "A",<br>"num": 10 } |
| Nulo     | "telefone": null                         |

---

## Exemplo Completo

```
{  
  "nome": "João",  
  "idade": 30,  
  "emails": ["joao@email.com", "joao2@email.com"],  
  "ativo": true,  
  "endereco": {  
    "rua": "Rua das Flores",  
    "numero": 123  
  }  
}
```

---

## Prática em Dart (Flutter)

Vamos criar e manipular esse mesmo JSON dentro de um app Flutter.

### **Criar Map (equivalente ao JSON):**

```
Map<String, dynamic> usuario = {  
  "nome": "João",  
  "idade": 30,  
  "emails": ["joao@email.com", "joao2@email.com"],  
  "ativo": true,  
  "endereco": {  
    "rua": "Rua das Flores",  
    "numero": 123  
  }  
};
```

### **Converter Map para JSON:**

```
import 'dart:convert';  
  
void main() {  
  String jsonString = json.encode(usuario);  
  print(jsonString);  
}
```

### **Saída:**

```
{"nome":"João","idade":30,"emails":["joao@email.com","joao2@email.com"],  
,"ativo":true,"endereco":{"rua":"Rua das Flores","numero":123}}
```

### **Converter JSON para Map (deserialização):**

```
void main() {  
  String jsonString = '{"nome":"João","idade":30}';  
  Map<String, dynamic> usuario = json.decode(jsonString);  
  print(usuario["nome"]); // João  
}
```

---

## Exercício Prático Sugerido

1. Criar um objeto JSON no Dart com:
  - nome
  - idade
  - lista de hobbies
  - cidade
  - status (ativo ou não)
2. Exibir esse objeto no console como JSON (usando `json.encode()`).
3. Criar uma string JSON e convertê-la em Map para acessar um campo específico (usando `json.decode()`).

---

## Resumo Rápido

| Conceito | Descrição                                                                                             |
|----------|-------------------------------------------------------------------------------------------------------|
| JSON     | Formato leve de troca de dados (texto)                                                                |
| Chave    | Sempre string entre aspas                                                                             |
| Valor    | Pode ser string, número, booleano, array, objeto ou null                                              |
| Dart     | Usa <code>Map&lt;String, dynamic&gt;</code> para representar JSON                                     |
| Pacote   | <code>dart:convert</code> faz a conversão com <code>json.encode()</code> e <code>json.decode()</code> |

## 2. JSON com SharedPreferences

## 2. JSON com SharedPreferences

**Tópico:** Armazenamento de pequenas configurações e preferências

### Objetivo do Tópico

Apresentar como utilizar o pacote `shared_preferences` em conjunto com JSON para **armazenar dados simples e rápidos localmente**, como:

- Nome de usuário
  - Tema do app (claro/escuro)
  - Preferências salvas
  - Configurações
- 

### O que é o `shared_preferences`?

O pacote `shared_preferences` fornece uma forma **persistente e leve** de armazenar **dados simples** no dispositivo do usuário.

Ele grava informações como **pares chave-valor** de forma **síncrona com o armazenamento nativo** do Android/iOS.

*Ele é ideal para armazenar pequenas quantidades de dados, que não precisam de estrutura complexa.*

---

### Tipos de dados que podem ser armazenados diretamente:

- `int`
- `double`
- `bool`
- `String`

- `List<String>`

---

## Quando usar com JSON?

Quando você quiser armazenar **um objeto complexo** (ex: um usuário com nome, email, idade), você precisa **converter o objeto em JSON** e armazenar como `String`.

---

## Adicionando o pacote ao projeto

No `pubspec.yaml`:

```
dependencies:  
  shared_preferences: ^2.2.0
```

---

## Exemplo Prático: Salvando e carregando configurações

### 1. Criar um mapa com os dados:

```
Map<String, dynamic> configuracoes = {  
  "temaEscuro": true,  
  "volume": 0.8,  
  "usuario": "Lucas"  
};
```

### 2. Salvar usando `SharedPreferences`:

```
import 'dart:convert';  
import 'package:shared_preferences/shared_preferences.dart';  
  
void salvarConfiguracoes(Map<String, dynamic> config) async {  
  final prefs = await SharedPreferences.getInstance();  
  String jsonString = json.encode(config);  
  prefs.setString('config', jsonString);  
}
```

```
}
```

### 3. Carregar os dados:

```
Future<Map<String, dynamic>> carregarConfiguracoes() async {  
  final prefs = await SharedPreferences.getInstance();  
  String? jsonString = prefs.getString('config');  
  if (jsonString != null) {  
    return json.decode(jsonString);  
  }  
  return {}; // Retorna objeto vazio se não encontrar  
}
```

---

### Teste com Widget Flutter

```
class ConfigPage extends StatefulWidget {  
  @override  
  _ConfigPageState createState() => _ConfigPageState();  
}  
  
class _ConfigPageState extends State<ConfigPage> {  
  bool temaEscuro = false;  
  
  @override  
  void initState() {  
    super.initState();  
    carregar();  
  }  
  
  void carregar() async {  
    Map<String, dynamic> config = await carregarConfiguracoes();  
    setState(() {  
      temaEscuro = config['temaEscuro'] ?? false;  
    });  
  }  
}
```

```

void salvar() {
    salvarConfiguracoes({"temaEscuro": !temaEscuro});
    setState(() {
        temaEscuro = !temaEscuro;
    });
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text("Configurações")),
        body: Center(
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                    Text("Tema escuro: ${temaEscuro ? 'Ativo' : 'Inativo'}"),
                    ElevatedButton(
                        onPressed: salvar,
                        child: Text("Alternar Tema"),
                    )
                ],
            ),
        ),
    );
}

```

---

## Vantagens do uso com JSON

| Vantagem | Descrição                                             |
|----------|-------------------------------------------------------|
| Compacto | Permite salvar objetos como string                    |
| Simples  | Ideal para configurações, perfil do usuário, temas    |
| Rápido   | Acesso instantâneo, sem necessidade de banco de dados |
| Offline  | Funciona mesmo sem internet                           |



---

## Limitações

- Não é recomendado para grandes quantidades de dados
- Não suporta objetos complexos diretamente (sem conversão)
- Armazenamento em string, sem criptografia por padrão (dados sensíveis devem ser evitados)

---

## Atividade Prática Sugerida

**Desafio em dupla:** Criar uma tela de preferências onde o usuário possa:

- Escolher se o tema é escuro ou claro
- Informar o nome e salvar
- Exibir os dados salvos ao abrir o app

Armazene esses dados como JSON dentro do SharedPreferences.

---

## Resumo

| Conceito                   | Explicação                                              |
|----------------------------|---------------------------------------------------------|
| SharedPreferences          | Armazena pares chave-valor simples                      |
| JSON + String              | Permite armazenar objetos como string                   |
| <code>json.encode()</code> | Serializa ( $\text{Map} \rightarrow \text{String}$ )    |
| <code>json.decode()</code> | Desserializa ( $\text{String} \rightarrow \text{Map}$ ) |
| Ideal para                 | Configurações, tema, perfil do usuário                  |

### 3. JSON com Arquivos Locais (path\_provider)

### 3. JSON com Arquivos Locais (path\_provider)

**Tópico:** Criação, leitura e escrita de arquivos .json no dispositivo

---

#### Objetivo do Tópico

Mostrar como criar, ler e salvar arquivos .json localmente no dispositivo do usuário utilizando Flutter, permitindo a **persistência de dados estruturados offline**, como:

- Listas de produtos
  - Dados de tarefas
  - Perfis de usuário
  - Histórico de ações
- 

#### Pacotes Utilizados

- `path_provider`: localiza diretórios padrão no dispositivo (documentos, cache, etc.)
  - `dart:io`: biblioteca nativa do Dart para manipulação de arquivos
  - `dart:convert`: para converter `Map` ↔ JSON
- 

#### Adicionando ao `pubspec.yaml`

```
dependencies:  
  flutter:  
    sdk: flutter  
  path_provider: ^2.1.2
```

---

## Por que usar arquivos .json?

- Permite **persistência estruturada** dos dados.
  - Suporta **listas e objetos complexos**.
  - Funciona **offline**, ideal para apps sem acesso constante à internet.
  - Fácil de **ler/modificar/exportar** se necessário.
- 

## Etapas para trabalhar com arquivos JSON

1. Obter o caminho do diretório local com `path_provider`
  2. Criar ou acessar o arquivo `.json`
  3. Escrever dados convertidos com `json.encode()`
  4. Ler dados e converter com `json.decode()`
- 

## Exemplo: Lista de tarefas armazenada em um arquivo .json

### 1. Importações necessárias:

```
import 'dart:io';  
import 'dart:convert';  
import 'package:path_provider/path_provider.dart';
```

---

### 2. Obter o arquivo local:

```
Future<File> _getArquivo() async {  
  final diretorio = await getApplicationDocumentsDirectory(); //  
  diretório local  
  final caminho = '${diretorio.path}/tarefas.json';
```

```
    return File(caminho);  
}
```

---

### 3. Salvar lista como JSON:

```
Future<void> salvarTarefas(List<Map<String, dynamic>> lista) async {  
    final arquivo = await _getArquivo();  
    String jsonTarefas = json.encode(lista); // converte para String  
    await arquivo.writeAsString(jsonTarefas); // salva no arquivo  
}
```

---

### 4. Ler e converter os dados do arquivo:

```
Future<List<Map<String, dynamic>>> lerTarefas() async {  
    try {  
        final arquivo = await _getArquivo();  
        String conteudo = await arquivo.readAsString();  
        List<dynamic> dados = json.decode(conteudo);  
        return dados.cast<Map<String, dynamic>>();  
    } catch (e) {  
        return []; // retorna lista vazia se erro (arquivo não existe,  
etc)  
    }  
}
```

---

## Widget exemplo simples

```
class ListaTarefasPage extends StatefulWidget {  
    @override  
    _ListaTarefasPageState createState() => _ListaTarefasPageState();  
}  
  
class _ListaTarefasPageState extends State<ListaTarefasPage> {  
    List<Map<String, dynamic>> tarefas = [];
```

```

@override
void initState() {
  super.initState();
  lerTarefas().then((lista) {
    setState(() {
      tarefas = lista;
    });
  });
}

void adicionarTarefa(String titulo) {
  final nova = {"titulo": titulo, "concluida": false};
  setState(() {
    tarefas.add(nova);
  });
  salvarTarefas(tarefas);
}

void remover(int index) {
  setState(() {
    tarefas.removeAt(index);
  });
  salvarTarefas(tarefas);
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("Minhas Tarefas")),
    body: ListView.builder(
      itemCount: tarefas.length,
      itemBuilder: (_, index) {
        return ListTile(
          title: Text(tarefas[index]["titulo"]),
          trailing: IconButton(
            icon: Icon(Icons.delete),
            onPressed: () => remover(index),

```

```
        ),
      );
    },
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () => adicionarTarefa("Nova Tarefa"),
    child: Icon(Icons.add),
  ),
);
}
```

---

## Estrutura típica do JSON salvo

```
[
  {
    "titulo": "Estudar Flutter",
    "concluida": false
  },
  {
    "titulo": "Revisar aula de JSON",
    "concluida": true
  }
]
```

---

## Vantagens

| Vantagem    | Descrição                                      |
|-------------|------------------------------------------------|
| Offline     | Funciona sem internet                          |
| Estruturado | Armazena listas e objetos                      |
| Legível     | Pode ser lido/editado manualmente              |
| Portável    | Fácil de exportar para backup ou sincronização |

---

## Limitações

| Limitação                                | Solução                       |
|------------------------------------------|-------------------------------|
| Lento para grandes volumes               | Usar banco como SQLite        |
| Não indicado para dados muito sensíveis  | Usar criptografia             |
| Pode gerar erro se o arquivo não existir | Usar tratamento com try-catch |

---

## Atividade Prática Sugerida

### Desafio individual:

Criar um app com as funcionalidades:

- Campo de texto para cadastrar produtos (nome, valor)
  - Botão "Salvar"
  - Listar produtos salvos
  - Armazenar tudo em um arquivo produtos.json
- 

## Resumo

| Etapas                 | Ação                                                |
|------------------------|-----------------------------------------------------|
| 1. Obter diretório     | <code>getApplicationDocumentsDirectory()</code>     |
| 2. Criar/abrir arquivo | <code>File(path)</code>                             |
| 3. Escrever            | <code>file.writeAsString(json.encode(dados))</code> |



|        |                                               |
|--------|-----------------------------------------------|
| 4. Ler | <code>json.decode(file.readAsString())</code> |
|--------|-----------------------------------------------|